

Bank Management System

Project Proposal & Architectural Specification

Course Code: CSE282 — Section: 04 — Semester: Summer 2026

Repository: github.com/TasmeoulMiyad/Bank-Management-System

Team Members:

- Tasmeoul Miyad (Lead) — 2024100000181
- Md Shawon Chowdhury Setu — 2024100000238
- Md Farhatul Hasan Rabbi — 2024100000411
- Miraj Masud Munna — 2024100000521
- Rifat Ahamed — 2024000000064

1. Project Introduction & Objective

The **Bank Management System** is a desktop-based banking workstation application built in Java. It models daily commercial banking operations: opening accounts, crediting balance-validated deposits, processing rule-constrained withdrawals, executing atomic cross-account transfers, and persisting ledger records to local storage. The software addresses the risks of spreadsheet discrepancies and manual ledger tampering by encapsulating business rules directly into an Object-Oriented domain hierarchy.

1. Account Onboarding

Register Savings and Current accounts with customer profiles and distinct financial parameters.

2. Guarded Withdrawals

Enforce minimum balance floors or overdraft ceilings via custom business exceptions.

3. Inter-Account Transfers

Atomic two-phase transactions that balance debits and credits between valid accounts.

4. Audit Ledger & Search

Auto-incremented, timestamped transaction tracking with fast multi-parameter lookups.

2. Problem Statement & Solution

Manual record keeping and spreadsheet tracking in local cooperative banks or training environments introduce operational friction:

Rule Non-Compliance

Spreadsheets fail to prevent account liquidations below required minimum reserves.

Inconsistent Auditing

No auto-generated timestamp tokens exist to track individual teller operations chronologically.

Asymmetric Fund Transfers

Manual entry can deduct funds from a sender without successfully crediting the recipient.

Heavy Database Overhead

Traditional commercial banking tools require complex network DBMS daemons unsuitable for standalone desks.

Solution: The system provides an offline desktop application written in Java SE that guarantees invariant preservation (overdraft ceilings, balance floors) through polymorphic domain methods and custom exception handling, while persisting data to structured flat files.

3. OOP Concept Implementation

The project directly demonstrates all four pillars of Object-Oriented Programming:

Encapsulation

State attributes (balance, accountNumber, customerId) are strictly private. State alterations require explicit gatekeeper methods like deposit() and deductBalance().

Abstraction

Account is declared an abstract class that enforces an abstract withdrawal contract: public abstract void withdraw(double amt), hiding policy details.

Inheritance

SavingsAccount and CurrentAccount extend Account, reusing common transaction lists, balance tracking, and customer bindings.

Polymorphism

withdraw() is dynamically dispatched at runtime to evaluate either the savings balance floor or the current overdraft ceiling during transfers.

4. Class Structure & Design

The system enforces a clean separation between domain models, the business controller, and the presentation layer:

Class Name	Key Variables / Fields	Key Methods / Functions
Account (Abstract)	accountNumber, balance, customer, transactions	deposit(), deductBalance(), addTransaction(), withdraw() (abstract)
SavingsAccount	minimumBalance	withdraw() (enforces balance floor), getMinimumBalance()
CurrentAccount	overdraftLimit	withdraw() (enforces overdraft limit), getOverdraftLimit()
Customer	customerId, name, phone, email	getDisplayInfo(), setPhone(), setEmail(), getters
Transaction	counter, transId, type, amount, date	getDetails() (formats chronological audit log string)
Bank	accounts, FILE_NAME	addAccount(), findAccount(), transfer(), saveToFile(), loadFromFile()
BankGUI	bank, displayArea	createAccountDialog(), depositDialog(), withdrawDialog(), transferDialog(), saveAndExit()

5. Data, Variables & I/O Specification

Core Entities

- **Account:** Account ID, balance, transactions.
- **Customer:** ID, name, phone, email.
- **Transaction:** Counter, ID, timestamp, amount.

GUI Inputs

- **Account Dialog:** Type, ID, opening deposit.
- **Cash Actions:** Target account, amount.
- **Transfer:** Sender ID, receiver ID, amount.

System Outputs

- Scrollable ledger with account records.
- Chronological audit log with tokens.
- Modal dialogs for confirmations/errors.

6. Exception Handling Strategy

All business constraints are verified before commit. If an operation breaches safety policies, the domain layer throws a descriptive exception caught by the GUI layer to prevent abnormal termination:

Minimum Balance Breach

Thrown when a savings account debit drops balance below the required floor: $B - amt < minBalance$.

Invalid Account Reference

Thrown during lookup or transfers if sender or recipient account cannot be resolved.

Overdraft Ceiling Exceeded

Thrown when a current account debit exceeds the agreed facility: $B - amt < -overdraftLimit$.

Number Format Exception

Caught in GUI dialogs when alphabetical characters are entered into numeric fields.

7. System Architecture & Class Relationships

The structural hierarchy, association links, and exception flows are organized as follows:



8. GUI Layout (Java Swing)

The interface uses a standard desktop layout running on the Swing Event Dispatch Thread (EDT):

- **North Header Panel:** Banner titled "Bank Management System".
- **West Control Panel:** An 8-button responsive GridLayout managing workflows: 1. *Create Account*, 2. *Deposit Funds*, 3. *Withdraw Funds*, 4. *Inter-Account Transfer*, 5. *Search Records*, 6. *Transaction Ledger*, 7. *View All Accounts*, 8. *Save & Exit*.
- **Center Canvas:** Scrollable, non-editable JTextArea displaying query outputs, ledger statements, and transfer receipts formatted with monospace alignments.
- **Modal Dialogs:** User input capture and non-fatal exception feedback rendered via JOptionPane dialogs.

9. Conclusion

The Bank Management System is a targeted software engineering project that satisfies all core requirements of the CSE282 laboratory. It applies the four primary OOP paradigms through an abstract account hierarchy, incorporates reliable text persistence via delimiter streams, provides structured exception handling for balance and transaction safety, and presents a responsive Java Swing interface.